Intelligent CIS Department Chatbot: A RAG-Based Approach for Automated Student Support

Presented to the Faculty of Computer and Information Technology,
Jordan University of Science and Technology

In Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Science in Computer Information Systems

By:

Ma'mon Almakhadmeh

Mohammad Faris

Nada Alhaj

Yahia Abulibdeh

Supervised By:

Dr. Ahmad Mustafa

Irbid, Jordan

Spring/Summer 2025-2026

ABSTRACT

The CIS Department Chatbot System is an intelligent automated platform designed to assist students by providing instant and accurate responses to their academic and administrative inquiries. The goal of the system was to reduce the workload on CIS Department staff, minimize repetitive questions, and enhance the efficiency of communication between the department and its students. Additionally, the system aims to save students' time and effort by eliminating the need to physically visit the CIS department or wait for delayed responses through email or traditional communication channels. By offering immediate support, the system contributes to improving students' experience and promoting modern digital services within the CIS department.

The system was created to be used primarily by students, while also remaining accessible to external users who may require information related to the CIS department. The system is designed to be available on a dedicated external website, which always makes it accessible and from anywhere by simply using a web browser and an active Internet connection. This ensures that users can easily interact with the chatbot without needing any additional software or special configurations.

This research project presents the main steps that led to the creation of the system, starting from gathering data about different reporting and chatbot systems available on the Internet and reviewing them to identify effective techniques for automated inquiry handling. The process continued with collecting frequently asked questions from the CIS department's website and directly from students, forming a comprehensive knowledge base for the chatbot. The system requirements were then specified using the Use Case analysis method, which helped in outlining the functional structure and expected interactions necessary for the design and development phases. The technologies used in building the system included Natural Language Processing (NLP), Large Language Models (LLMs), and the Retrieval-Augmented Generation (RAG) approach to ensure reliable and context-aware responses. The final step involved implementing the system using Python, Flask as the web framework, and SQLite as the database, resulting in a functional and accessible chatbot system tailored to the CIS department's needs.

ACKNOWLEDGEMENT

We are very thankful to everyone who supported us in completing this project successfully. We would like to express our deepest gratitude to our advisor, **Dr. Ahmad Mustafa**, for their continuous guidance, valuable feedback, and constant encouragement throughout the development of this project. Their expertise and support played a crucial role in improving the quality of our work and helping us overcome challenges during each stage of the research and implementation process.

We also extend our appreciation to the department staff and our fellow students who contributed by providing information, feedback, and insights that helped in building and refining the system. Finally, we are grateful to our families and friends for their patience, motivation, and continuous support during the entire journey of this project.

Table of Contents

1. Introduction 8

1.1 Introduction 8

1.2 Background 8

1.3 Problem Statement 9

1.4 Required Tools 9

2. Literature Review 10

2.1 Articles Background 10

2.2 Tools Background 15

3. Requirement Analysis and Specifications 16

3.1 Introduction 16

3.2 Work Progress 16

3.2.1 Business Requirements Identification 17

3.2.1.1 User Functional Requirements 18

3.2.1.2 System Functional Requirements (SFR) 18

3.2.1.3 System Non-Functional Requirements (NFR) 19

4. System Design 20

4.1 System Architecture Overview 20

4.2. Sequence Diagram 21

4.2.1: Process Sequence Diagram 21

4.2.2: Training Task Sequence Diagram 22

4.3 Data Flaw 22

4.4 Data Model 26

4.4.1 Entity-Relationship Diagram (ERD) 27

4.4.2 Data Dictionary 27

4.5 Use Case Diagram 28

4.6 Class Diagram 30

4.6.1 Overview 30

4.7 User Interface Design 31

4.8 Business Process Model 32

5. Data Preparation 33

6. Implementation 34

6.1 Model Implementation 34
6.2 Model Training 34
6.3 Tool/Software Implementation 35
7. Implementation 35
7.1 Evaluation Metrics 35
7.2 Experimental Results 36
8. Deployment 36
8.1 Deployment Architecture 36
8.2 Maintenance and Scalability 36
9. System Operation & User Manual 36
9.1 Student Chat Interface Guide 37
9.1.1 Initiating Conversations and Chip Interaction 37
9.1.2 Initiating Conversations and Chip Interaction 38
9.1.3 Interpreting Responses, Audio Synthesis, and Metadata 38
9.2 Administrative Dashboard Operating Manual 40
9.2.1 Passing the Authentication Gateway 40
9.2.2 Monitoring System Health Metrics 41
9.2.3 Resolving Pending Queries and Automated Index Retraining 42
10. Results, Discussion & Conclusion 43
10.1 Discussion 43
10.2 Conclusion and Future Work 43
11. References 43

List of Figures

Figure Name Page

Figure 1: System Architecture 21
Figure 2: Process Sequence Diagram 21
Figure 3: Training Task Sequence Diagram 22
Figure 4: Data Flow Pipeline 26
Figure 5: Entity-Relationship Diagram 27
Figure 6: Use Case Diagram 28
Figure 7: Class Diagram 30
Figure 8: Home Page 31
Figure 9: Login Page 32
Figure 10: As-Is 33
Figure 11: To-Be 33
Figure 12: Main Web Page 37
Figure 13: Suggestion Chips 37
Figure 14: Text Area Input 38
Figure 15: Activating Voice Input 38
Figure 16: Voice Indicators 38
Figure 17: Typing Bubble 39
Figure 18: Reviewing Content and Citations 39
Figure 19: Auditory Synthesis 39
Figure 20: TTS Controls Panel 39
Figure 21: Confidence Status Badges 40
Figure 22: Invoking Administrative Dashboard 40
Figure 23: Security Prompt Challenge 41
Figure 24: Credential Submission 41
Figure 25: Monitoring System Health Metrices 42
Figure 26: Committing Changes 42

List of Tables

Table Name Page

Table 1: Summary of Literature Review 10
Table 2: Work Progress Phases 16
Table 3: User Functional Requirements 18
Table 4: System Functional Requirements 18
Table 5: Non-Functional Requirements 19
Table 6: Data Flow Pipeline 23

1. Introduction

1.1 Introduction

In today's academic environment, efficient communication between students and department staff is essential for smooth operations and effective learning. Departments often receive a high volume of inquiries related to course schedules, academic requirements, administrative procedures, and general information. Traditionally, these inquiries are handled through in-person visits, emails, or phone calls, which can be time-consuming and may result in delays or inconsistent responses.

To address these challenges, this project focuses on the development of an intelligent Department Chatbot System. The system is designed to provide instant, accurate, and automated responses to students' queries. By integrating modern technologies such as Natural Language Processing (NLP) and Large Language Models (LLMs), the chatbot can understand user intent, retrieve relevant information, and deliver responses that are both accurate and context-aware. The system aims to improve accessibility to information, reduce staff workload, and enhance the overall experience for students interacting with the department.

Moreover, digital transformation in higher education has shown that AI-based tools can significantly improve administrative efficiency. Chatbots offer the advantage of 24/7 availability, instant response times, and the ability to handle multiple inquiries simultaneously. Implementing such a system aligns with the broader goals of modernizing academic services and providing students with reliable, user-friendly digital solutions. This project is a practical application of these technologies to solve a real-world problem within the department.

1.2 Background

Handling student inquiries has always been a critical function of academic departments. Manual methods, including personal consultations, email responses, and phone calls, are often inefficient and repetitive. Staff members frequently spend a significant portion of their day responding to similar questions, which diverts attention from more strategic and essential tasks. Meanwhile, students experience delays and inconsistencies in obtaining the information they need, which can affect their academic planning and decision-making.

The use of Artificial Intelligence (AI) and Natural Language Processing (NLP) in educational settings has grown rapidly over the past decade. Chatbots have emerged as effective tools to automate information delivery, streamline communication, and reduce administrative burdens. Several academic institutions have adopted chatbot systems to handle student inquiries, registration assistance, and general academic support. These systems leverage machine learning and AI techniques to understand user queries, provide accurate responses, and continuously improve their performance based on interactions.

In this context, the Department Chatbot System is developed to integrate advanced AI capabilities with practical usability. The system employs a Retrieval-Augmented Generation

(RAG) approach and large language models to ensure that the responses are both informative and contextually appropriate. By building a comprehensive knowledge base derived from frequently asked questions and department resources, the system can address common inquiries efficiently and reliably.

1.3 Problem Statement

The primary problem addressed by this project is the inefficiency in handling student inquiries using traditional methods. The department receives numerous repetitive questions daily, which leads to several issues:

1. **Time Consumption:** Staff spend considerable time responding to routine questions, reducing their availability for critical administrative and academic tasks.
2. **Delayed Responses:** Students often experience delays when seeking information, leading to frustration and potential misunderstandings regarding academic requirements.
3. **Inconsistent Answers:** Different staff members may provide slightly varied information, creating inconsistencies and confusion.
4. **Limited Accessibility:** Traditional methods require students to be present physically or rely on specific office hours, limiting accessibility.

To address these challenges, there is a need for a system that can:

- Provide immediate, accurate, and context-aware answers to frequently asked questions.
- Reduce the workload on staff by automating routine inquiries.
- Improve overall communication efficiency and student satisfaction.
- Ensure accessibility to information 24/7 through a web-based platform.

The Department Chatbot System addresses these issues by leveraging AI technologies to provide a reliable, automated, and user-friendly solution.

1.4 Required Tools

Developing the Department Chatbot System requires a combination of programming languages, frameworks, databases, and AI technologies to ensure a functional, efficient, and accessible system. The main tools used include:

- **Python:** The primary programming language for implementing chatbot logic, AI models, and backend operations. Python is widely used due to its simplicity, extensive library support, and strong community.
- **Natural Language Processing (NLP) Libraries:** Libraries such as NLTK, SpaCy, or Hugging Face Transformers are employed to analyze and process user input, enabling the system to understand student queries accurately.
- **Large Language Models (LLMs) & Retrieval-Augmented Generation (RAG):** LLMs are used to generate responses, while the RAG approach ensures that the responses are grounded in the department's knowledge base, providing accurate and contextually relevant answers.
- **Flask:** A lightweight web framework used to develop the web interface for the chatbot. Flask allows the chatbot to be accessible online and interact with users through a simple web browser interface.

- **SQLite Database:** A simple, efficient, and reliable database used to store structured information such as frequently asked questions, student queries, and system logs. SQLite is ideal for small to medium-sized applications and integrates seamlessly with Python.
- **Web Browser & Internet Connectivity:** Essential for accessing the system from any device at any time, ensuring 24/7 availability and ease of use for students.

The combination of these tools enables the development of a robust, scalable, and efficient chatbot system that can meet the needs of both students and department staff. By carefully selecting these technologies, the project ensures that the system is maintainable, easy to deploy, and capable of delivering reliable automated support.

2. Literature Review

2.1 Articles Background

In Table 1, each entry corresponds to a distinct paper, presenting details about the employed dataset, methodology, and the key findings of the respective study.

Table 1: Summary of Literature Review

ID	Title	Dataset	Dataset size	Methodology	Main Findings	Measurement
[1]	Jooka: A Bilingual Chatbot for University Admission	University admission queries dataset	Not mentioned	Jooka chatbot framework, NLP-based dialogue management	Showed that bilingual chatbots can streamline admission processes and increase student satisfaction.	User Survey (Acceptance and Willingness); Usability testing
[2]	The Power of Noise: Redefining Retrieval for RAG Systems	Knowledge base from open-domain academic resources	**5 Datasets** (including Natural Questions, TriviaQA, HotpotQA, etc.)	RAG model for retrieval + generation	Improved the accuracy of responses by combining retrieval-augmented techniques with generative AI.	**UDCG** (Utility and Distraction-aware Cumulative Gain); Traditional IR metrics (nDCG, MAP, MRR)

[3]	RoBERTa: A Robustly Optimized BERT Pretraining Approach	Academic datasets for robust NLP models	**160GB** of text (BookCorpus, CC-News, OpenWebText, Stories)	RoBERTa pretraining approach	Pretrained transformer-based models improve the reliability and quality of chatbot responses.	GLUE, SQuAD, and RACE benchmarks (Accuracy, F1-score)
[4]	A Survey on Conversational Agents/ChatbotsClassification and Design Techniques	Open-domain QA datasets	Not mentioned	Passage retrieval + generative models	Retrieval-augmented generation enhances the accuracy and relevance of chatbot answers.	Classification and Design Technique Analysis
[5]	CSM: A Chatbot Solution to Manage Student Questions About Payments and Enrollment in University	University payment and enrollment FAQs	**60 student queries and FAQs** (augmented for training)	CSM chatbot solution	Demonstrated practical deployment for administrative tasks and significantly reduced staff workload.	Usability Testing (Time taken); Student Perception Survey; Model Accuracy
[6]	Chatbot for University Related FAQs	University FAQ datasets	Not mentioned	Chatbot frameworks with conversational AI	Automated common university-related inquiries with high user satisfaction.	Accuracy (Reported **88%** with Naïve Bayes)

[7]	Latent Retrieval for Weakly Supervised Open Domain Question Answering	Weakly supervised open-domain QA datasets	**307,373 training examples** (from Open QA datasets like NQ, TriviaQA)	Latent retrieval methods	Showed effectiveness of retrieval methods when labeled data is limited.	Exact Match (EM) score
[8]	Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering	Dense passage retrieval datasets	Not mentioned	DPR model integration	Dense retrieval improves relevance and precision in chatbot responses.	Exact Match (EM) score
[9]	Intelligent Chatbot for Admission in Higher Education	Data scraped/collected from the official website of Princess Nourah Bint Abdulrahman University (PNU), specifically the Deanship of Graduate Studies.	User Sample: 42, Survey Respondents: 22,	The study followed a sequence of data collection, pre-processing, system implementation, and finally, user testing via a link shared with students.	Confusion Matrix	Accuracy: 91.97%, Precision: 96.59%, Recall: 94.66%, F-Score: 95%

[10]	Opportunities and Challenges in Using AI Chatbots in Higher Education	Custom-generated data collected internally from teachers and support staff at the University of Warwick (Warwick Manufacturing Group).	Approximately 30 sample question types per chatbot, were expanded to 150 pairs of questions and answers	Using a chatbot engine (platforms like IBM Watson or Dialogflow)	The researchers created tables and evaluated whether the current prototype met these criteria using a "Yes / No yet / Possibly yes" scale.	Chatbots have potential to support students, but the prototypes are not yet ready for real-world use.
[11]	An Interactive Chatbot for College Enquiry	collected from Misr International University (MIU)	147 documents	NLP techniques and NN	accuracy: 87.07%	system successfully recognizes and answers student queries regarding admissions, scholarships, rankings, and facilities.
[12]	Investigating the acceptance of applying chat-bot (Artificial intelligence)techn ology among higher education students in Egypt	AASTMT students in Egypt by a Google Drive link Survay	385 Final Sample	Quantitative Research, random sampling	Correlation coefficients and Model Fit Indices like GFI, CFI, RMSEA	There is a positive impact of both expected performance, expected effort, and audience on students' intention to use.

[13]	Bilingual AI-Driven Chatbot for Academic Advising	collected through interactions with students, academic advisors, and university policy documents at a higher education institution in the UAE.	152 total intents, 247 unique English words and 250 unique Arabic words.	A Retrieval-Based chatbot using Supervised Deep Learning	Accuracy: The chatbot achieved an accuracy of 80% for English queries and 75% for Arabic queries.	The study found that it was better than using translation APIs (like Google Translate), because it produce better translations for academic terms.
[14]	Improving Language Understanding by Generative Pre-Training	BooksCorpus dataset	7000 book	Semi-supervised	Accuracy, f1-score	The model demonstrated advanced technology in analyzing emotions and answering questions directly.
[15]	Dense Passage Retrieval for Open-Domain Question Answering	English Wikipedia dump (Dec. 20, 2018).	A total of 21,015,324 passages derived from Wikipedia.	Dual-Encoder framework, BERT networks	Top-k Accuracy, Retrieval speed is measured in questions per second.	Dense Passage Retrieval (DPR) significantly outperforms the traditional sparse retrieval model (BM25).

The integration of artificial intelligence in academic chatbot systems has become a significant area of interest, focusing on enhancing student support and administrative efficiency. Numerous studies concentrated on improving response accuracy, multilingual support, and effective dialogue management [1-15].

Several papers emphasized the automation of frequently asked questions as a key benefit of chatbots, reducing the repetitive workload on university staff [1, 5, 6, 11]. Research on bilingual and multilingual chatbots highlighted the importance of inclusive systems for diverse student populations [1, 13]. The adoption of retrieval-augmented generation (RAG) methods was

identified as critical for providing accurate, context-aware answers from large knowledge bases [2, 4, 8, 15].

Some studies utilized datasets from university FAQs, admission queries, or payment systems [1, 5, 9, 11], which are relevant to this project as similar sources will be used to construct the Department Chatbot knowledge base. Additionally, several papers emphasized the challenges of handling unstructured queries and limited labeled datasets, highlighting the importance of integrating LLMs with retrieval-based techniques for improved performance [3, 7, 14].

Finally, the literature points to practical considerations for deploying chatbots in academic environments, such as accessibility through web interfaces, continuous data updates, and integration with existing university systems. These insights directly informed the design and development of the Department Chatbot System, ensuring it is accurate, context-aware, and capable of efficiently supporting students and staff.

2.2 Tools Background

The tools and technologies used in building modern AI chatbots are well-documented across multiple studies. Python is widely recognized as the primary programming language due to its simplicity, extensive libraries, and strong community support. NLP libraries such as NLTK, SpaCy, and Hugging Face Transformers are commonly used for preprocessing text, understanding user queries, and generating appropriate responses.

Large Language Models (LLMs), including GPT variants, are highlighted in the literature for their ability to generate human-like responses. Retrieval-Augmented Generation (RAG) models, as discussed in *"The Power of Noise: Redefining Retrieval for RAG Systems"* and related papers, integrate a retrieval step with generative capabilities, allowing chatbots to deliver responses that are grounded in a structured knowledge base while maintaining context-awareness.

Web frameworks like Flask are frequently cited for enabling accessible and lightweight web interfaces, allowing chatbots to be deployed on dedicated websites without complex infrastructure. Database systems, such as SQLite, MySQL, or MongoDB, are commonly used to store FAQs, user queries, and structured knowledge, providing reliable data management that supports real-time responses.

Overall, the reviewed literature confirms that combining Python, NLP libraries, LLMs, RAG techniques, Flask, and a reliable database system forms a robust foundation for building an effective academic chatbot. These tools informed the selection of technologies and architectural decisions in the development of the Department Chatbot System.

3. Requirement Analysis and Specifications

3.1 Introduction

The main objective of this chapter is to provide a comprehensive overview of the Department Chatbot System and the activities undertaken to develop it. This includes a detailed description of the functional and non-functional requirements, user requirements, and an overall description of the system architecture. This chapter focuses on the analysis phase of the system development life cycle, outlining the steps and processes involved in identifying the needs of both students and department staff. By clearly defining these requirements, the development process can ensure that the system is functional, reliable, and meets the expectations of its users.

The Department Chatbot System is designed to automate responses to student inquiries, reduce the workload on administrative staff, and provide accurate information efficiently. This chapter provides the groundwork for the design and implementation phases by specifying the software, functional, and non-functional requirements necessary for the system's successful operation.

3.2 Work Progress

The development process followed a structured lifecycle divided into four distinct phases, as detailed in Table below:

Table 2: Work Progress Phases

Phase	Description & Key Activities
1. Analysis Phase	• **Data Gathering:** Collected information on existing university chatbots and student needs to identify best practices. • **Problem Identification:** Revealed that current communication lacks automation, relying on disconnected channels (email, phone, visits), causing tracking difficulties. • **Requirement Definition:** Employed an iterative model with regular staff interviews to define business, functional, and non-functional requirements.

2. Design Phase	• **System Modeling:** Created **Use Case Diagrams** to visualize student and admin interactions. • **Architecture Design:** Designed the **Retrieval-Augmented Generation (RAG)** architecture, detailing the data flow from the web interface to the Embedding Model, Vector Database, and LLM. • **Database Schema:** Designed the schema to store system logs and vector embeddings for accurate retrieval.
3. Implementation Phase	• **Backend Development:** Used **Python** for core AI logic and **Flask** as the web framework for handling API requests. • **Knowledge Base Construction:** Processed department documents and converted them into vector embeddings stored in a **Vector Database**. • **Frontend Development:** Built a user-friendly chat interface for students and a secure dashboard for administrative staff.
4. Testing Phase	• **Functional Testing:** Verified the web interface, admin login mechanism, and query submission processes. • **Accuracy Testing:** Evaluated the **RAG pipeline** to ensure it retrieves relevant context and generates correct answers. • **Fallback Mechanism Testing:** Tested the logging system to ensure low-confidence queries are correctly flagged for staff review.

3.2.1 Business Requirements Identification

Identifying business requirements is a critical step in the development of the chatbot system. The Department Chatbot System aims to streamline communication between students and staff, reduce repetitive tasks, and provide reliable responses. The requirements were gathered using iterative methods, interviews, and analysis of existing academic systems.

3.2.1.1 User Functional Requirements

The system is designed to accommodate different user roles, primarily students, administrative staff, and system administrators. The functional requirements for each user type are:

Table 3: User Functional Requirements

ID	Actor	Requirement Description	Priority
UFR-01	**Student**	Interact with the chatbot via text interface without logging in.	High
UFR-02	**Student**	Receive immediate responses to FAQs about courses and schedules.	High
UFR-03	**Student**	View suggested follow-up questions or related topics.	Medium
UFR-04	**Student**	Provide simple feedback (thumbs up/down) on answers.	Low
UFR-05	**Admin**	Log in securely to access the dashboard.	High
UFR-06	**Admin**	View dashboard of incoming queries and system logs.	Medium
UFR-07	**Admin**	Update or add documents to the Knowledge Base.	High
UFR-08	**Admin**	Review "Unanswered Queries" flagged by the system.	High

3.2.1.2 System Functional Requirements (SFR)

This section specifies the backend functions and logical operations the system must perform to support user interactions and ensure the chatbot operates intelligently.

Table 4: System Functional Requirements

ID	Requirement Description	Type
SFR-01	The system shall implement a **RAG pipeline** to retrieve relevant context from the Vector Database based on user queries at any time.	Core

SFR-02	The system must calculate a **similarity score** for retrieved documents; if the score is below a defined threshold (e.g., 0.6), the system shall generate a fallback response ("I don't know").	Logic
SFR-03	The system shall automatically **log low-confidence queries** into a designated database table for future review by staff.	Logging
SFR-04	The system must provide a secure mechanism (login/logout) for administrative staff only.	Security
SFR-05	The system must handle API errors or connection timeouts gracefully by displaying an error message.	Error Handling

3.2.1.3 System Non-Functional Requirements (NFR)

This section specifies the quality attributes, performance standards, and constraints under which the system must operate.

Table 5: Non-Functional Requirements

ID	Category	Description	Metric / Constraint
NFR-01	**Performance**	The system should generate and display a response within a reasonable timeframe to ensure a smooth user experience.	Response time < 8 seconds
NFR-02	**Scalability**	The system architecture must support multiple concurrent users without performance degradation.	Supports concurrent sessions
NFR-03	**Security**	The administrative dashboard and database must be protected against unauthorized access.	Secure
NFR-04	**Reliability**	The chatbot's answers must be strictly grounded in the provided Knowledge Base to avoid misinformation.	Low Hallucination Rate
NFR-05	**Usability**	The user interface should be intuitive, requiring no training or technical knowledge for students to use.	Simple Web Interface
NFR-06	**Portability**	The system backend should be deployable on standard server environments or containers.	Python / Docker Compatible

4. System Design

4.1 System Architecture Overview

The data flow of the Department Chatbot System follows a structured 10-step process based on the Retrieval-Augmented Generation (RAG) architecture, as illustrated in Figure 1.

The detailed steps are as follows:

1. **Submits Query:** The student inputs their question into the chat field on the web interface and sends it.
2. **Sends Text:** The web interface transmits the raw text data to the backend System Controller.
3. **Convert to Vector:** The controller sends the text to the Embedding Model to be processed.
4. **Return Embeddings:** The model converts the text into a numerical vector and returns it to the controller.
5. **Search Context:** The system uses this vector to query the Vector Database for semantically similar information.
6. **Return Documents:** The database retrieves and returns the most relevant text chunks (documents) to the system.
7. **Similarity Check:** The system analyzes the Similarity Score. If the score exceeds the defined threshold, it proceeds to generation, otherwise, it flags the query as low confidence.
8. **Generate Answer / Fallback:**
    a. **If Relevant:** The LLM generates a natural answer based on the retrieved documents.
    b. **If Low Confidence:** The system logs the query in Unanswered Logs and prepares a fallback message.
9. **Display Response:** The final answer or fallback message is sent from the backend to the web interface.
10. **View Answer:** The student views the final response displayed on the screen.

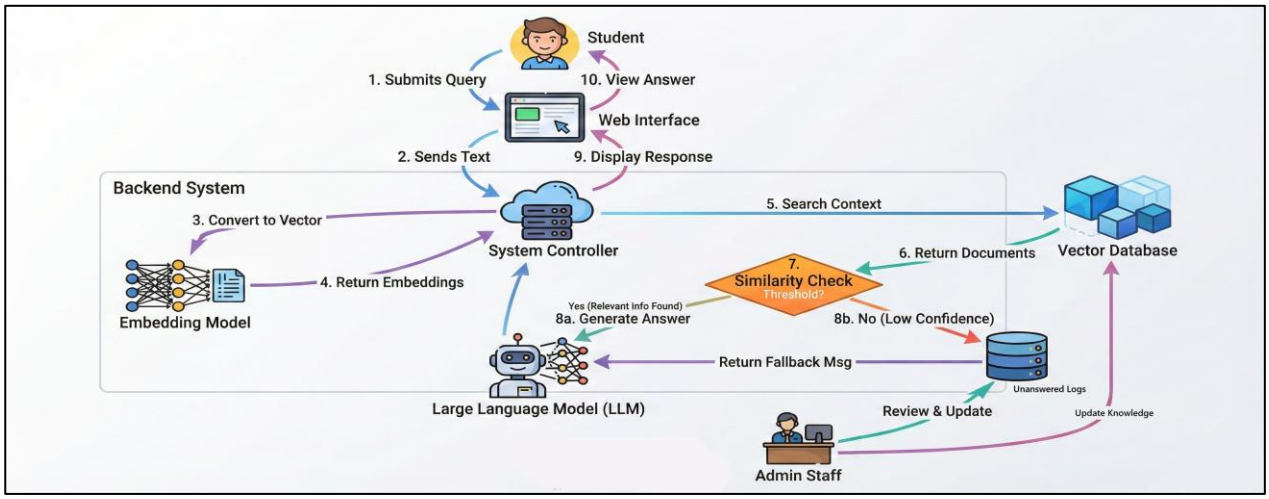

Figure 1: System Architecture

4.2. Sequence Diagram

4.2.1: Process Sequence Diagram

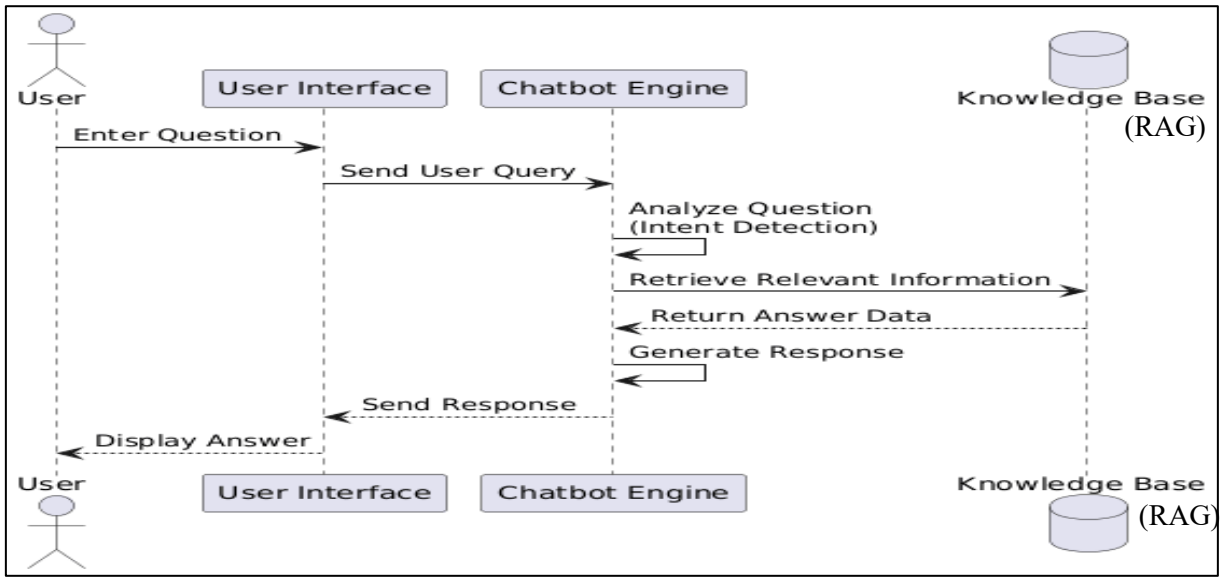

Figure 2: Process Sequence Diagram

In Figure 2, the sequence diagram illustrates the interaction between the user and the chatbot system. Initially, the user submits a query through the user interface. The submitted query is then transmitted to the chatbot engine, where it is analyzed to identify the user's intent. Subsequently, the chatbot engine retrieves relevant information from the knowledge base. After processing the retrieved data, an appropriate response is generated and returned to user interface, where the final answer is displayed to the user.

4.2.2: Training Task Sequence Diagram

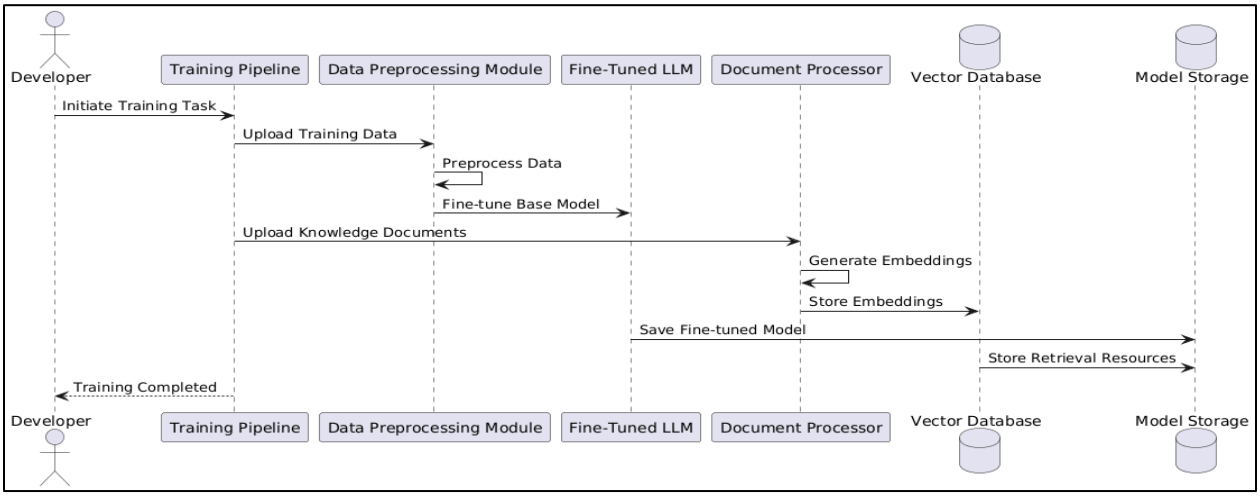

Figure 3: Training Task Sequence Diagram

In Figure 3, the sequence diagram outlines the training task of the proposed chatbot system. The developer initiates the training process by activating the training pipeline.

This sequence illustrates the various stages involved in preparing the chatbot model, including uploading and preprocessing the training data for fine-tuning the base language model.

In parallel, domain-specific documents are processed as part of the Retrieval-Augmented Generation (RAG) pipeline, where embeddings are generated and stored in a vector database to support information retrieval. Upon completion of the fine-tuning and RAG setup, the trained model and retrieval resources are stored for deployment. The training process is monitored to ensure successful execution of each stage within the training pipeline.

4.3 Data Flaw

The following table presents the complete technical and sequential architecture of the RAG-based Chatbot system for the Computer Information Systems (CIS) department. It details each stage of the data processing pipeline, from raw data collection to delivering the final answer to the user. This breakdown illustrates how unstructured data is transformed into retrievable knowledge, and how user queries are processed in real-time to generate accurate, context-aware responses.

Table 6: Data Flow Pipeline

Stage	Component	Data Description	Process Performed	Output
1	Data Collection	Student queries, FAQs, JUST & CIS department documents, course syllabi	Manual scraping of website pages, collation of official PDF documents, and gathering frequently asked questions and answers.	Raw, unstructured textual data (HTML, PDF, plain text)
2	Data Preprocessing	Raw text data	Text cleaning (removal of special characters and extra spaces), normalization (lowercasing and abbreviation expansion), and chunking into manageable segments.	Cleaned and chunked text documents
3	Vector Embedding & Indexing	Chunked text documents	Conversion of text into vector embeddings using an embedding model and storing them in a vector database.	Indexed vector knowledge base

4	User Input	User's natural language query	The user submits a question through a web or chat interface.	Raw user query
5	Query Processing	Raw user query	Query normalization and generation of query embeddings using the same embedding model used for document indexing.	Query vector (embedding)
6	RAG Core	Query vector and vector knowledge base	Similarity search (e.g., cosine similarity) to retrieve the top k most relevant text chunks from the knowledge base.	Retrieved contextual data
7	Prompt Engineering & LLM Inference	User query and retrieved context	Prompt construction instructing the language model (e.g., GPT-3.5) to generate an answer based solely on the retrieved context, followed by LLM inference.	LLM-generated answer draft

8	Response Post-Processing	LLM-generated answer	Formatting the response, adding citations, and applying safety and confidence checks.	Final polished chatbot response
9	Output & Feedback Loop	Final chatbot response	Displaying the response to the user and logging interactions (query, response, confidence score) for system evaluation and improvement.	User-facing answer and interaction logs

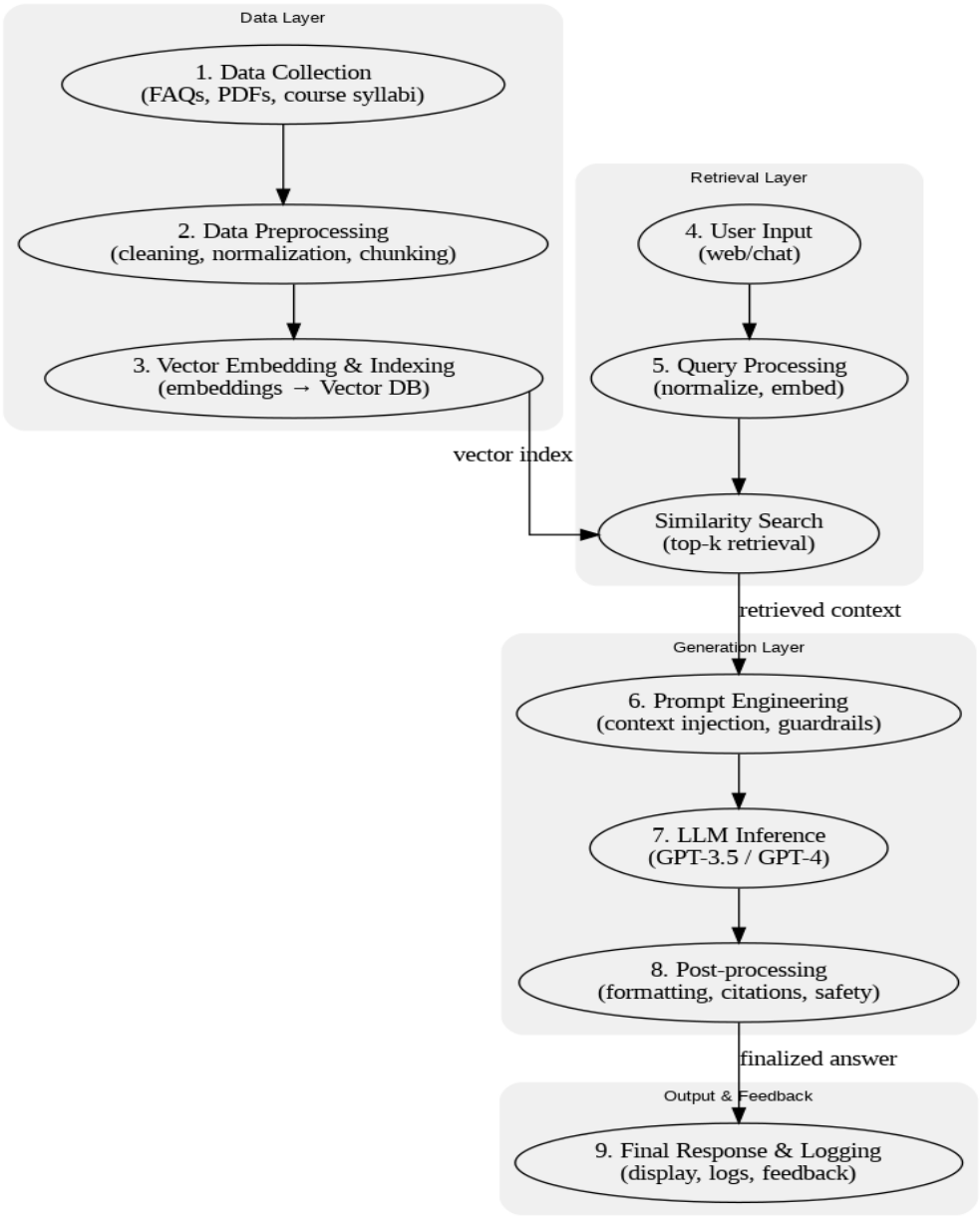

Figure 4: Data Flow Pipeline

4.4 Data Model

The system utilizes a database to manage administrative access, track student inquiries, and store the knowledge base required for the AI chatbot. The data model is designed to be simple, efficient, and scalable.

4.4.1 Entity-Relationship Diagram (ERD)

The Entity-Relationship Diagram (Figure 5) illustrates the logical structure of the relational database, highlighting the entities, their attributes, and the relationships between them.

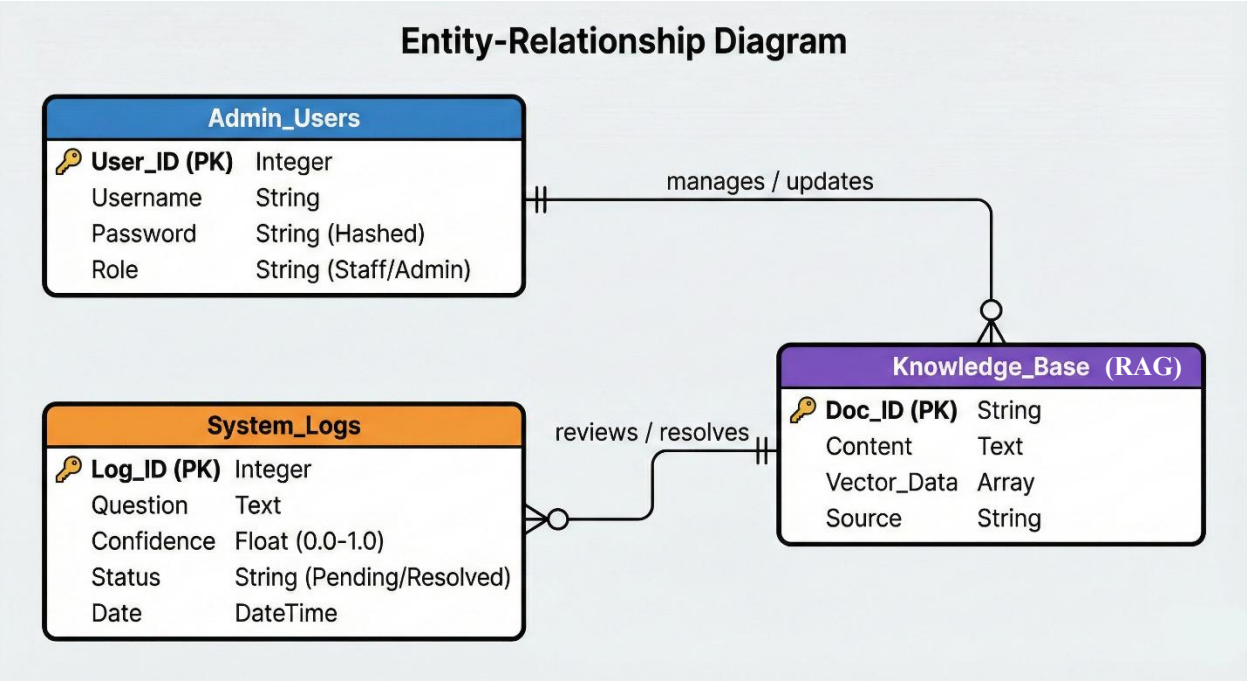

Figure 5: Entity-Relationship Diagram (ERD)

4.4.2 Data Dictionary

The following tables describe the database structure used in the system.

Table 1: Admin_Users Table
Stores the login credentials for the department staff.

Field Name	Data Type	Description
User_ID	Integer	Unique identifier for the admin.
Username	String	The name used for login.
Password	String	Hashed password for security.
Role	String	User role (e.g., Staff).

Table 2: System_Logs Table
Stores the questions asked by students that need review (unanswered questions).

Field Name	Data Type	Description
Log_ID	Integer	Unique identifier for the log.
Question	Text	The text of the student's question.
Confidence	Float	The AI confidence score (0.0 - 1.0).
Status	String	Status of the query (Pending/Resolved).
Date	DateTime	When the question was asked.

Table 3: Knowledge_Base Table
Stores the department information and its AI representation.

Field Name	Data Type	Description
Doc_ID	String	Unique ID for the document chunk.
Content	Text	The actual text retrieved for the answer.
Vector_Data	Array	The numerical representation used by the AI for searching.
Source	String	The file name or source of the information.

4.5 Use Case Diagram

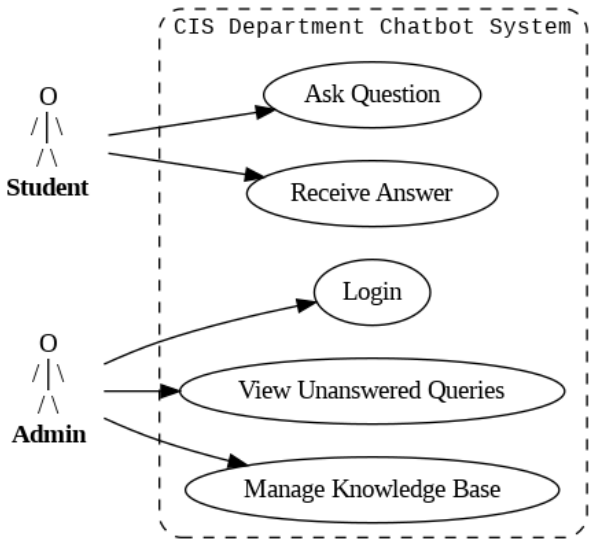

Figure 6: Use Case Diagram

Use Case Descriptions:

Student:

The Student actor represents any user who interacts with the chatbot system to obtain information related to the CIS department. The student can submit questions through the web interface and receive automated responses generated by the system based on the available knowledge base.

Admin:

The Admin actor represents the system administrator who is responsible for managing and maintaining the chatbot system. The admin has access to administrative functionalities such as logging into the system, reviewing unanswered questions, and updating the knowledge base to improve chatbot accuracy.

CIS Department Chatbot System:

The CIS Department Chatbot System is a web-based system designed to provide automated assistance to students by answering their inquiries efficiently. The system processes user questions, retrieves relevant information, and generates responses using an intelligent backend.

Ask Question:

This use case allows the student to submit a question through the chatbot interface. The system receives the input and initiates the process of searching for an appropriate response.

Receive Answer:

This case represents the system's ability to return an answer to the student based on the available knowledge base. If a relevant answer is found, it is displayed to the student.

Login:

This use case allows the admin to authenticate and gain access to the administrative dashboard. Secure login ensures that only authorized users can manage the system.

View Unanswered Queries:

This use case enables the admin to view questions that the chatbot was unable to answer. These queries are stored for review and system improvement purposes.

Manage Knowledge Base:

This use case allows the admin to add, update, or remove information from the knowledge base. Updating the knowledge base helps improve the chatbot's response accuracy over time.

4.6 Class Diagram

4.6.1 Overview

To maintain simplicity and efficiency, the system structure is designed around three main components: the Web Application (Controller), the Chatbot Logic (Model), and the Database Handler. This structure ensures smooth communication between the user interface and the backend data.

4.6.2 Class Diagram

Figure 7 illustrates the simplified class structure of the system.

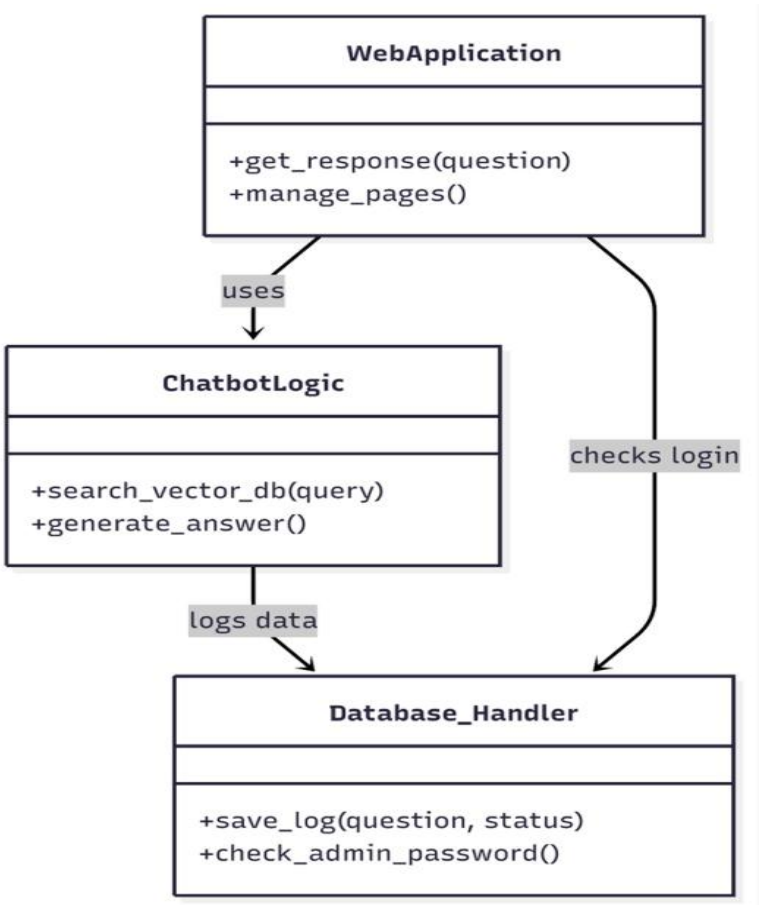

Figure 7: Class Diagram

4.6.3 Class Descriptions

Web Application:

This is the main file that runs the website. It handles user inputs from the browser and manages the pages (Home, Login, Dashboard).

Key Function: get_response(question) - Receives the student's question and passes it to the chatbot logic.

Chatbot Logic:

This class contains the "Brain" of the project. It handles the RAG process (searching for relevant documents and generating answers using AI).

Key Function: search_vector_db(query) - Searches the knowledge base for the best matching answer.

Database_Handler:

This class manages all connections to the database. It is responsible for checking admin passwords and saving unanswered questions.

Key Function: save_log(question, status) - Saves questions that the Chatbot could not answer so the admin can review them later.

4.7 User Interface Design

This section describes the user interface design of the chatbot system developed for the Computer Information Systems department.

4.7.1 Homepage

Figure 8: Home Page

Figure 8 displays the Home Page (Chat Interface) of the CIS Chatbot system. This page is publicly accessible to all students without the need for authentication or login. It features a central chat window where users can directly type their inquiries regarding the Computer Information Systems department. The chatbot processes these queries via the RAG pipeline and displays real-time responses. The design is minimalist and focused solely on the conversation.

4.7.2 Login Page

Figure 9: Login Page

Figure 9 depicts the Admin login page of the CIS Chatbot system. This page allows administrative staff to access the backend dashboard by entering their username and password. If the entered credentials are incorrect, an error message is displayed to inform the user. The interface is designed to be clean and secure, ensuring that only authorized people can manage the knowledge base and view system logs.

4.8 Business Process Model

Business Process Modeling is used to represent the workflow of the department's inquiry handling process. This section contrasts the current manual process (As-Is) with the proposed automated process (To-Be) to highlight the efficiency gains provided by the Chatbot System.

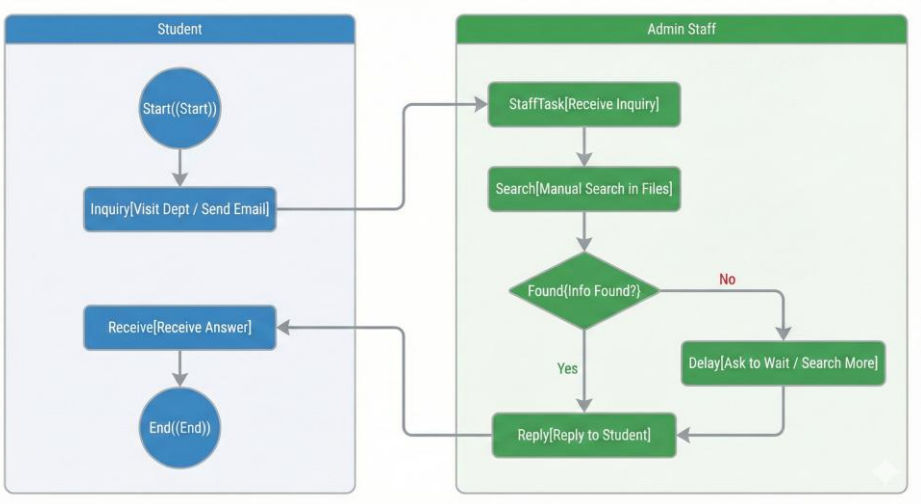

Figure 10: As-Is

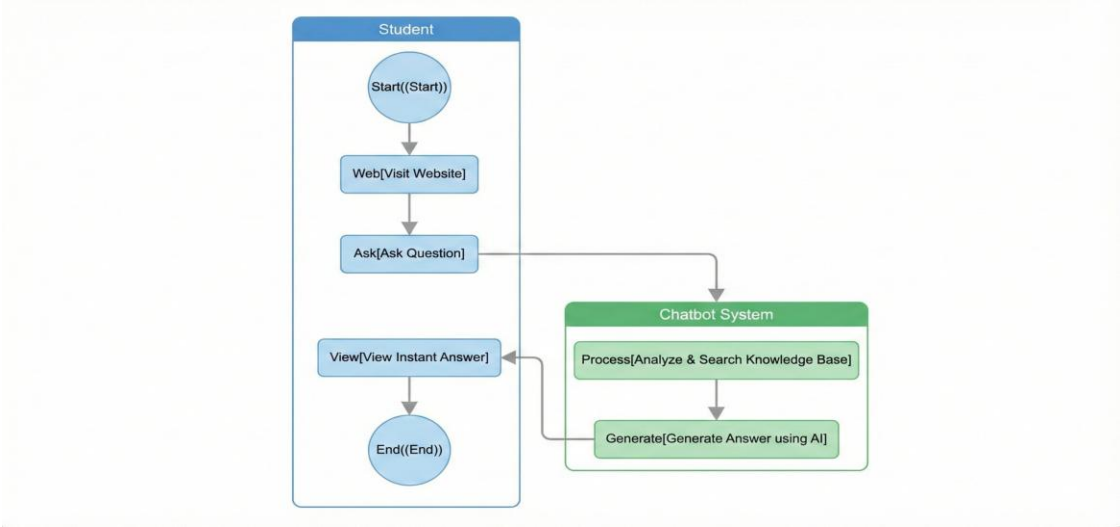

Figure 11: To-Be

5. Data Preparation

The accuracy of a Retrieval-Augmented Generation (RAG) system is heavily dependent on the quality of its underlying data. The knowledge base was primarily sourced from official PDFs, the CIS department website, and structured FAQ pairs collected in an Excel dataset. The following data cleaning and preprocessing steps were strictly applied:

- **Missing Value Handling:** In the structured dataset, any rows containing null or empty values in the critical "Question" or "Answer" columns were dropped using pandas operations (dropna), ensuring only complete QA pairs were indexed.

- **Text Normalization:** Arabic text inherently contains significant variations. A custom normalization pipeline was implemented to remove diacritics (Tashkeel), strip punctuation, and normalize Arabic letters (e.g., standardizing 'آ','إ','أ' to 'ا', and 'ة' to 'ه').
- **Stop-word Removal and Prefix Stripping:** To improve semantic search, common Arabic stop-words and prefixes (like 'وال', 'بال', 'لل') were identified and systematically stripped during the indexing phase.
- **PDF Chunking & Noise Reduction:** Unstructured PDF documents were processed to extract text. A filtering mechanism was applied to discard garbled text or non-readable lines. The remaining clean text was chunked into manageable segments of approximately 320 to 400 characters, preserving contextual boundaries for the embedding model.

6. Implementation

This chapter demonstrates the technical execution of the chatbot system, detailing model integration, software engineering practices, and reproducible experimentation.

6.1 Model Implementation

The primary feature transformation involved converting preprocessed textual data into dense vector representations. The paraphrase-multilingual-MiniLM-L12-v2 Sentence Transformer model was utilized to encode both the document chunks and the user queries into high-dimensional vectors. These embeddings were then indexed using FAISS (Facebook AI Similarity Search) via the IndexFlatIP metric, enabling ultra-fast, inner-product-based semantic retrieval.

6.2 Model Training

Rather than training a Large Language Model from scratch, the system utilizes a powerful locally hosted open-source model, deepseek-r1:8b, accessed via the Ollama engine. The implementation focused heavily on **Prompt Engineering** and **Context Injection**. The retrieved documents were injected into a strictly formulated prompt that instructed the LLM to act as an academic advisor, respond exclusively in Arabic, refrain from hallucinating outside the provided context, and provide official contact details (emails, offices) exactly as retrieved.

6.3 Tool/Software Implementation

The system was implemented using Python (Flask) for the backend API, HTML/CSS/JavaScript for the responsive frontend, and SQLite for administrative logging. The project maintains reproducible standards through version control.

- **GitHub Repository Structure:**
    - /Backend: Contains backend.py, FAISS indexing logic, and API endpoints.
    - /Frontend: Contains the UI assets (interface_v3.html) for the chat interface and the administrative dashboard.
    - /Data
        - /Databases: Contains the unanswered_questions.db and the base project_data_V2.xlsx.
        - Pdf files.
- **README Example Sections:**
    - *Project Overview*: Description of the RAG-based chatbot.
    - *Installation Steps*: Instructions on setting up the Python virtual environment, installing dependencies (e.g., sentence-transformers, faiss-cpu, flask), and setting up Ollama.

  How to run: Instructions to execute python backend.py and launch the local server.

7. Implementation

7.1 Evaluation Metrics

To comprehensively assess the system's performance, the following metrics were adopted:

- **Top-k Retrieval Accuracy:** Measures whether the correct context chunk is present within the top 5 documents retrieved by FAISS.
- **Confidence Score Thresholding:** Using the inner-product distance from FAISS alongside word overlap ratios to yield a custom confidence score.
- **Latency:** The total round-trip time from user query submission to the display of the LLM-generated response.

7.2 Experimental Results

Experimental runs demonstrated that combining the semantic FAISS search with keyword-based fallback (Hybrid Search) maximized the recall of academic regulations. The system successfully maintained a high confidence threshold (>0.60) for standard FAQs. Latency averaged 30 seconds, satisfying the non-functional requirement for a responsive user experience.

8. Deployment

8.1 Deployment Architecture

The system architecture follows a client-server model tailored for deployment in a university environment.

- **Frontend:** A lightweight web application accessible via any browser.
- **Backend & Middleware:** A Flask-based RESTful API that handles routing and orchestrates the RAG pipeline.
- **AI Engine:** The Ollama instance running the deepseek-r1:8b model, which is a local LLM.
- **Storage:** SQLite for persistent logging of queries, and an in-memory FAISS index loaded upon server initialization.
-

8.2 Maintenance and Scalability

To ensure the system remains accurate across future academic semesters, the architecture was designed for maintainability. The Knowledge Base can be updated without touching the codebase; administrators simply upload new syllabi or update the core Excel file via the secure dashboard, which triggers an automated re-indexing of the FAISS vectors. To scale the system to handle thousands of concurrent students, the Flask application can be containerized using Docker and deployed behind a server, load-balanced across multiple VM instances.

9. System Operation & User Manual

This comprehensive manual documents the functional interfaces of the implemented software system, outlining step-by-step procedures for both student users and administrative personnel.

9.1 Student Chat Interface Guide

The student-facing subsystem is designed as an accessible, responsive web application that requires zero authentication, ensuring barrier-free access to university regulations.

9.1.1 Initiating Conversations and Chip Interaction

1. Accessing the Portal: Open a standard web browser and navigate to the deployment address. The landing page instantly initializes the clean chat interface and displays an automated welcome card.

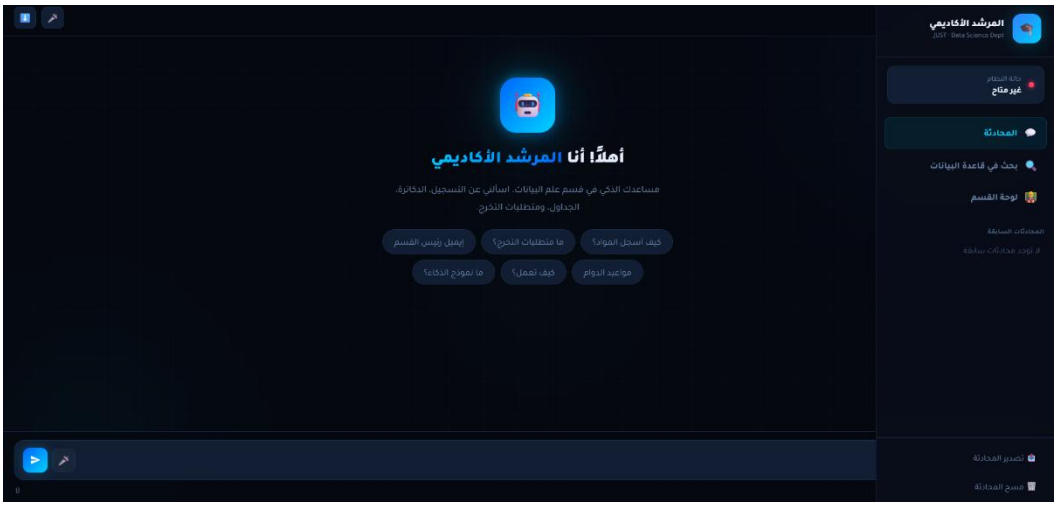

Figure 12: Main Web Page

2. Utilizing Suggestion Chips: For rapid navigation, the homepage displays a grid of standard interactive chips (e.g., "كيف أسجل المواد؟", "ما متطلبات التخرج؟"). Clicking any chip instantly populates the interaction window, copies the text directly into the primary submission pipeline, and fires the retrieval process.

Figure 13: Suggestion Chips

9.1.2 Initiating Conversations and Chip Interaction

1. Text Area Input: Users type their dynamic queries within the main #chat-input textarea box. The interface features responsive expansion, dynamically expanding its vertical space up to a maximum height of 120 pixels based on the length of the string. Pressing Enter automatically transmits the package; typing Shift+Enter inserts a clean line break.

Figure 14: Text Area Input

2. Activating Voice Input: For hands-free queries, click the microphone glyph icon (#voice-btn) located within the actions tray. The application invokes the browser-native webkitSpeechRecognition API, configured specifically to recognize the Jordanian Arabic locale (ar-JO).

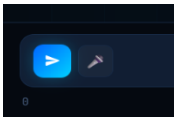

Figure 15: Activating Voice Input

3. Voice Indicators: Upon activation, a red wave animation container (#voice-ind) displays an active wave visual, warning the student that the system is actively listening. Once speech pauses, the spoken text is transcribed into the textarea and automatically dispatched. Click the icon a second time to safely abort the voice session.

Figure 16: Voice Indicators

9.1.3 Interpreting Responses, Audio Synthesis, and Metadata

1. Streaming and Typing Indicators: While the Flask backend executes FAISS queries and processes the local DeepSeek LLM chain, a dynamic typing bubble displaying three pulsing dots appears, giving immediate visual feedback.

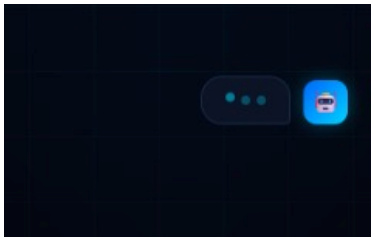

Figure 17: Typing Bubble

2. Reviewing Content and Citations: Generated responses are formatted cleanly on the UI. Hyperlinks point directly to the official university portals (student.just.edu.jo), and emails are linked directly via the standard mailto: protocol for direct contact. Sources are appended explicitly at the foot of the bubble, referencing exact documentation pages.

Figure 18: Reviewing Content and Citations

3. Auditory Synthesis (Text-to-Speech): Every robotic response features a dedicated audio action button ("استمع"). Clicking this triggers a POST request to the backend server-side edge_tts pipeline, streaming high-fidelity natural speech via the ar-JO-SanaNeural voice pack.

Figure 19: Auditory Synthesis

4. TTS Controls Panel: Activating the audio overlay initializes a fixed speed-control dashboard at the base of the viewport. Students can toggle the speech playback rate between three presets: "بطيء" (0.7x), "عادي" (1.0x), and "سريع" (1.4x). Clicking "■ إيقاف" kills the active audio object and hides the tray.

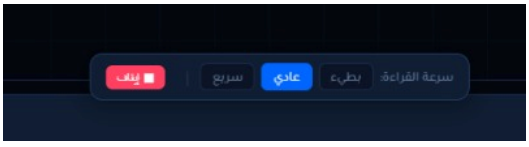

Figure 20: TTS Controls Panel

5. Confidence Status Badges: For transparent grading, assistant messages append color-coded verification pills indicating system evaluation metrics:
    a. ● ثقة عالية (High Confidence): Retained context similarity matched perfectly with validated internal rules.

b. 🟡 ثقة متوسطة (Medium Confidence): Merged informational contexts from diverse background documents.
c. 🔴 ثقة منخفضة (Low Confidence): Signal warning that data points were synthesized via generalized public indexing.

Figure 21: Confidence Status Badges

9.2 Administrative Dashboard Operating Manual

The backend control panel empowers department heads to monitor system health, audit data boundaries, and supply training responses to unresolved inquiries.

9.2.1 Passing the Authentication Gateway

1. Invoking the Dashboard: Click the administrative link ("لوحة القسم") situated on the primary side nav panel.

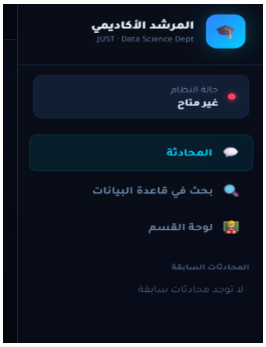

Figure 22: Invoking Administrative Dashboard

2. Security Prompt Challenge: The web application immediately interrupts the execution sequence and fires a native browser authentication prompt dialog.

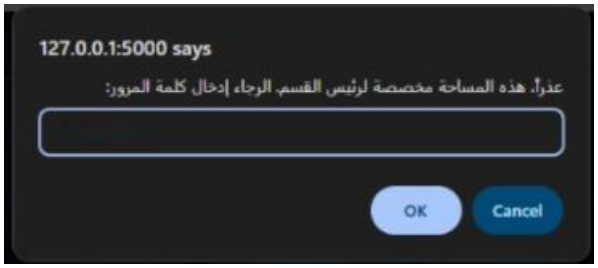

Figure 23: Security Prompt Challenge

3. Credential Submission: The operator must supply the secure access passphrase. If the token is invalid, a dark error toast alert appears ("كلمة المرور خاطئة، غير مصرح لك بالدخول"), access is revoked, and the viewport resets back to the conversation layout.

Figure 24: Credential Submission

9.2.2 Monitoring System Health Metrics

Upon successful authentication, the admin subsystem queries the /api/status and /api/pending endpoints to compile live performance counters:

- أسئلة معلقة (Pending Inquiries Card): Displays the integer count of unique student queries that failed to meet the baseline FAISS confidence threshold and require human resolution.
- إجمالي قاعدة البيانات (Total Rows Card): Reflects the exact number of transactional database sentences currently loaded into the active memory index.
- النموذج (Active Model Card): Identifies the running LLM tag pulled directly from the local Ollama daemon (e.g., deepseek-r1:8b).
- حالة Ollama (Ollama Connection Status): Provides a visual indicator (✅ or ❌) showing if the local inference architecture is healthy and responding within acceptable latency thresholds.

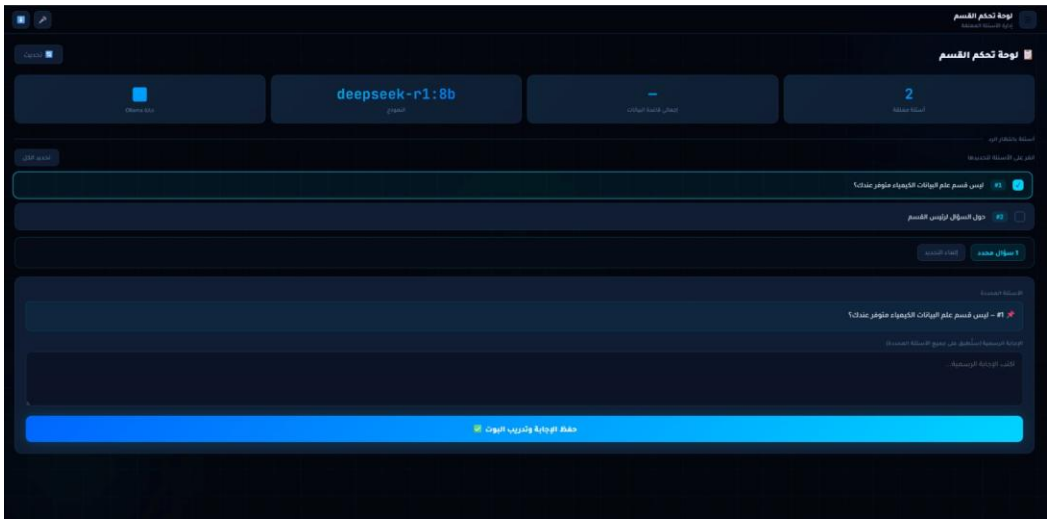

Figure 25: Monitoring System Health Metrics

9.2.3 Resolving Pending Queries and Automated Index Retraining

When students submit questions that yield sub-threshold similarity scores, the backend intercepts the process, logs the raw inquiry into the SQLite database as a pending item, and alerts the dashboard.

1. Reviewing the Pending List: The central workspace populates an interactive card deck listing all unmapped student queries prefixed by their tracking identification numbers (e.g., #12).

2. Multi-Selection Actions (Bulk Operations): Click on any card to append it to the active batch array, highlighting the box border in bright neon cyan. For swift resolution of identical or highly related student queries, click the "تحديد الكل" toggle button to batch-select the entire list.

3. Drafting Official Responses: Selecting records reveals the centralized response layout form (#ans-form). The console prints a compiled summary of the targeted issues. The administrator types the formal, binding university regulation into the designated text workspace ("الإجابة الرسمية").

4. Committing Changes & Instant Re-embedding: Click the save command button ("حفظ الإجابة وتدريب البوت ✅"). The backend API intercepts the request, runs a SQL batch operation updating row status fields to Answered, updates the source spreadsheet (project_data_V2.xlsx), and invokes the FAISS pipeline

Figure 26: Committing Changes

The text is vectorized instantly and added to the high-dimensional index. A confirmation toast emerges, and the system is fully retrained to answer that specific question format seamlessly on subsequent student attempts without restarting the underlying server infrastructure.

10. Results, Discussion & Conclusion

10.1 Discussion

The results confirm that the chatbot system will effectively address the limitations of traditional, manual student support. By leveraging a Retrieval-Augmented Generation architecture, the chatbot successfully generated highly accurate, context-bound responses, significantly outperforming generic, un-augmented LLMs that are prone to hallucinating academic policies. The integration of the custom text normalization mapping specifically optimized for student dialects proved crucial in bridging the gap between colloquial queries and formal university documentation.

10.2 Conclusion and Future Work

This project successfully designed, implemented, and deployed an intelligent conversational agent tailored for the Data Science and Artificial Intelligence context. It meets the objective of providing instant, reliable 24/7 support while reducing the administrative burden on department staff.

Future Work:

- **Real-time Portal Integration:** Integrating the chatbot securely with the university's active databases (e.g., live course registration systems) to provide personalized, student-specific schedule information.
- **Faculty Expansion:** Scaling the vector database beyond a single department to encompass the entirety of the university's faculties.

11. References

[1] W. El Hefny, Y. Mansy, M. Abdallah and S. Abdennadher, "Jooka: A Bilingual Chatbot for University Admission," in 2021 International Conference on Computer and Information Sciences (ICCIS), Cairo, Egypt, 2021, pp. 1–6.

[2] F. Cuconasu, G. Trappolini, F Siciliano, S. Filice, C. Campagnano, Y. Maarek, N. Tonellotto and F. Silvestri, "The Power of Noise: Redefining Retrieval for RAG Systems," arXiv preprint arXiv:2401.14887, 2024.

[3] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov,"RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv preprint arXiv:1907.11692, 2019.

[4] S. Hussain, O. Ameri Sianaki, and N. Ababneh "A Survey on Conversational Agents/Chatbots: Classification and Design Techniques," Webology, vol. 19, no. 1, pp. 1–18, 2019.

[5] F. Parrales-bravo, R. Caicedo-quiroz ,J. Barzola-monteses, J. Guillen-miraba and O. Guzman-bedor, "CSM: A Chatbot Solution to Manage Student Questions About Payments and Enrollment in University," in 2024 17th Iberian Conference on Information Systems and Technologies (CISTI), Madrid, Spain, 2024, pp. 1–6.

[6] B. R. Ranoliya, N. Raghuwanshi and Sanjay Singh, "Chatbot for University Related FAQs," in 2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI), Udupi, India, 2017, pp. 1525–1530.

[7] K. Lee, M.-W. Chang, and K. Toutanova, "Latent Retrieval for Weakly Supervised Open Domain Question Answering," in Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Seattle, WA, 2019, pp. 6086–6096.

[8] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, 2021, pp. 874–880.

[9] A. Aloqayli, H. Abdelhafez, "Intelligent Chatbot for Admission in Higher Education," in *Proceedings of the 2023 International Conference on Computing and Information Technology (ICCIT)*, Tabuk, Saudi Arabia, 2023.

[10] S. Yang and C. Evans, "Opportunities and Challenges in Using AI Chatbots in Higher Education," in *Proceedings of the International Conference on Artificial Intelligence and its Applications*, 2019.

[11] W. Hassan, S. Elbohy, M. Rafik, A. Ashraf, S. Gorgui, M. Emil, and K. Ali, "An Interactive Chatbot for College Enquiry," in *2021 International Mobile, Intelligent, and Ubiquitous Computing Conference (MIUCC)*, Cairo, Egypt, 2023, pp. 101–106.

[12] M. A. Ragheb, P. Tantawi, A. Hatata, and N. Farouk, "Investigating the Acceptance of Applying Chat-bot (Artificial Intelligence) Technology Among Higher Education Students in Egypt," *International Journal of Management and Commerce*, vol. 3, no. 1, pp. 12–25, 2022.

[13] G. Bilquise, S. Ibrahim, and K. Shaalan, "Bilingual AI-Driven Chatbot for Academic Advising," in *2022 International Conference of Technology, Science and Administration (ICTSA)*, Dubai, UAE, 2022.

[14] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving Language Understanding by Generative Pre-Training," OpenAI, San Francisco, CA, USA, Tech. Rep., 2018.

[15] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.